Denoising Autoencoder: Run Comparison & Mathematical Explanation

1. Setup

Each trajectory is $x \in \mathbb{R}^{2400}$ (T = 800 timesteps × 3 axes). The noisy input is $\tilde{x} = x + \varepsilon$, with $\varepsilon \sim \mathcal{N}(0, \sigma^2 I)$. The autoencoder computes

$$\hat{x} = f_\theta(\tilde{x}) = \mathrm{Dec}(\mathrm{Enc}(\tilde{x})), \qquad z = \mathrm{Enc}(\tilde{x}) \in \mathbb{R}^k$$

where k is the embedding size, trained to minimize MSE $\mathcal{L} = \mathbb{E}\|\hat{x} - x\|^2$. Because the input is noisy but the training target is clean, this is a **denoising** autoencoder — it must learn the map from a noisy point back onto the clean data manifold, not just reconstruct its own input.

2. Base Configuration

Fixed hyperparameters shared across all 9 runs below (only σ, embedding size, and epoch count were varied between runs, as shown in Section 3):

Hyperparameter	Value	Meaning
N_TRAJECTORIES	1000	Total trajectories in dataset
T	800	Timesteps per trajectory
NOISE_SIGMA	0.6 / 1.6 (varied)	Std dev of Gaussian noise added to each (x, y, z) point
EMBEDDING_SIZE	4 / 8 / 16 / 32 (varied)	Size of the AE's compressed latent representation
EPOCHS	20 / 100 / 300 / 400 (varied)	Max number of full passes over the training set
BATCH_SIZE	64	Trajectories per gradient update
LR	1e-3	Adam learning rate

TRAIN_SPLIT	0.6	Fraction of trajectories used for training vs. testing

3. All Runs Ranked

Rank	Run	σ (noise)	embedding k	epochs	Mean test MSE
1	21-30-15	0.6	**32**	400	**0.0067**
2	21-28-39	0.6	16	400	0.0074
3	21-21-50	0.6	8	300	0.0102
4	21-27-21	0.6	8	400	0.0220*
5	21-32-00	1.6	32	400	0.0199
6	21-34-53	1.6	8	400	0.0220*
7	21-25-35	0.6	**4**	400	0.0256
8	21-14-02	0.6	8	**20**	0.0326
9 (worst)	**21-19-15**	**0.6**	**8**	**100**	**0.1536**

*Runs 4 and 6 both display mean MSE 0.0220 in the plot label — effectively tied.

4. Why the Top 3 Runs Win — the Manifold / Bottleneck Argument

A clean spiral trajectory does not really have 2400 independent degrees of freedom. It lies on a low-dimensional manifold parameterized by a handful of physical quantities generated in `trajectory.py` — radius, pitch/frequency, phase offset, vertical drift rate, etc. Call this the **intrinsic dimension** d^* (plausibly single digits).

For a bottleneck autoencoder trained on data near a d^*-dimensional manifold, reconstruction error from denoising decomposes approximately as:

$$\mathbb{E}\|\hat{x} - x\|^2 \approx \underbrace{\sigma^2 \cdot \frac{d^*}{k}}_{\text{residual noise leaking through}} + \underbrace{C(k)}_{\text{manifold-fit / capacity bias}}$$

Term 1 — noise leakage, shrinks as k grows

This is the classic subspace-denoising result: projecting isotropic noise ε onto a k-dimensional subspace and back removes most of the noise energy *as long as* $k \geq d^*$. If $k < d^*$, the model cannot even represent all true signal directions, and is forced to conflate signal with noise in the directions it does keep.

This term explains the clean monotonic trend at fixed σ = 0.6:

> emb 4 (0.0256) → emb 8 (0.0102–0.0220) → emb 16 (0.0074) → emb 32 (0.0067)

Term 2 — nonlinear approximation bias

Even on *noise-free* data, too few latent units means $\mathrm{Dec}(\mathrm{Enc}(\cdot))$ cannot exactly represent the manifold, adding systematic error. This is exactly the structured, sinusoidal x/y residual pattern observed earlier at k = 8 (period matching the spiral's own oscillation) — versus the small, unstructured z-residual, since z is close to linear and trivially representable even at low k.

Why 16 and 32 are close, but both beat 4 and 8

The spiral's intrinsic dimension d^* is probably around 4–6. So:

- **k = 16, 32** sit comfortably above d^* → both are in the *noise-limited, not capacity-limited* regime, which is why their scores are close to each other (0.0067 vs 0.0074).
- **k = 4, 8** sit at or below d^* → they pay the capacity penalty $C(k)$ heavily, which is why they lag behind despite otherwise identical training.

Why capacity matters less at high σ

Going emb 8 → 32 helped a lot at σ = 0.6 (0.0220 → 0.0067, a 3.3× improvement) but barely at σ = 1.6 (0.0220 → 0.0199, only 1.1×). The formula explains this directly: once $k \geq d^*$, the first term $\sigma^2 d^*/k$ scales *linearly* in σ^2, so at high noise this term dominates the total error regardless of further increases to k — you become **noise-floor-limited** rather than capacity-limited.

5. Why Run #9 (emb 8, 100 epochs) Is the Worst — and Why It's Not Just "Undertrained"

Comparing loss curves: Run #9's train/test loss **plateaus around 0.15–0.2 by epoch ~20 and stays flat** for the remaining 80 epochs — it is not still descending, it is stuck. Run #3 (same k = 8, same σ = 0.6, just 300 epochs instead of 100) reaches loss ≈ 0.01 by epoch ~20 and keeps improving toward its asymptote. Same architecture, same noise level, same data distribution — the only difference is random initialization / minibatch order — yet one run converges to loss ≈ 0.01 and the other to loss ≈ 0.15: a **15× gap** from identical hyperparameters.

> This is the signature of a **non-convex optimization landscape**. The loss surface $\mathcal{L}(\theta)$ for a ReLU network is highly non-convex, and gradient descent from different initial weights θ_0 can converge to different local minima or plateaus:

$$\theta_0^{(a)} \xrightarrow{\text{SGD}} \theta^{*(a)}, \quad \mathcal{L}(\theta^{*(a)}) \approx 0.01 \qquad \text{vs.} \qquad \theta_0^{(b)} \xrightarrow{\text{SGD}} \theta^{*(b)}, \quad \mathcal{L}(\theta^{*(b)}) \approx 0.15$$

Two concrete mechanisms that produce exactly this signature

1. **Dead ReLU units.** If enough encoder neurons get pushed to negative pre-activation early (a large negative gradient step, or unlucky initialization), then $\text{ReLU}(z) = 0$ for all inputs, and $\frac{\partial\,\text{ReLU}}{\partial z} = 0$ there too — those units receive **zero gradient forever**. This permanently shrinks the network's effective capacity below its nominal $k = 8$, pushing it into the high-$C(k)$ regime described above — i.e. it behaves like a much smaller, more capacity-starved bottleneck.
2. **A poor-conditioning basin.** Adam with a fixed learning rate $\eta = 10^{-3}$ can land in a flat region of the loss surface (small gradient norm $\|\nabla_\theta \mathcal{L}\|$) that is not the global optimum. Without something to escape it (an LR schedule, restart, or extra capacity to route around it), training just sits there — consistent with the observed curve going flat rather than diverging or oscillating.

The MSE histogram's fat tail out to MSE ≈ 2–4 (versus ≤ 0.08 for healthy runs) is the fingerprint of this failure: a stuck or degenerate encoder maps many different noisy inputs toward similar latent codes (a collapsed region of latent space), so most trajectories reconstruct as roughly the "average" spiral rather than their specific instance. A few especially atypical trajectories then get catastrophically bad reconstructions, producing the long tail.

> **Practical implication:** since this failure mode is initialization/seed-dependent rather than hyperparameter-dependent, early stopping alone will not fix it if the plateau lasts longer than the patience window (it would simply trigger and lock in the bad minimum). The fix is either (a) a few

> random restarts, keeping the best result, or (b) a learning-rate warmup/schedule to help the optimizer escape shallow plateaus early in training.